

PROJECT PLAN

Revenue Forecasting Under Uncertainty

Build plan, architecture, and a two-person work split with Claude session prompts

A CRM forecasting engine that predicts 30 / 60 / 90 day revenue and explains exactly why the forecast changed.

TEAM

2 members, parallel tracks

TIMELINE

10 working days per member

COST

Zero. Fully free and open source.

HEADLINE FEATURE

Forecast change attribution

Read Parts 1 to 5 together. Then each member reads their own part and pastes their prompt.

How to read this document

Parts 1 to 6 are shared context and both members should read them before writing any code. Part 7 is for Member A, Part 8 for Member B. Part 9 is the working agreement. Part 10 holds a prompt each member pastes into their own Claude session so Claude has the full project context without being re-explained.

Part	Contents	Who reads it
1	What we are building, and why the normal way fails	Both
2	How the system works: three questions and the dice trick	Both
3	Architecture, tech stack, database schema	Both
4	The two pieces of real engineering	Both
5	What we ship: four screens and the metrics	Both
6	Dividing the work, and the contract between the halves	Both
7	Member A instructions, day by day	Member A
8	Member B instructions, day by day	Member B
9	Working together: syncs, checkpoints, repo layout, git rules	Both
10	Claude session prompts, ready to paste	Each their own
11	Risks, and what to cut if time runs short	Both

The single most important idea in this plan. The two halves are connected by one small file, `src/contract.py`, written together on Day 1 and then frozen. Member B builds against a fake version of it while Member A builds the real one. Because of this, neither person is ever waiting on the other.

Part 1 — What we are building

A sales team keeps a list of deals they hope to close. Today, most companies forecast revenue by taking each deal, multiplying its value by a fixed percentage attached to its stage, and adding everything up. One number comes out. That number is almost always wrong.

We are building a replacement. Our system looks at each deal and answers three questions honestly, then rolls the dice ten thousand times to produce a **range** of revenue for the next 30, 60 and 90 days. And every time it runs, it saves a record — so when the number changes next week, it can point at the exact deals and reasons that moved it.

Why the normal way fails

Imagine a CRM screen showing this:

Deal	Value	Stage	Stage probability
Acme	Rs 500,000	Proposal	60%
Globex	Rs 300,000	Demo	40%
Initech	Rs 200,000	Negotiation	80%

The standard forecast is $(500k \times 0.6) + (300k \times 0.4) + (200k \times 0.8) = \text{Rs } 580,000$. Simple, and broken for four reasons.

1. **The percentages are invented.** Someone decided years ago that "Proposal means 60%". It is the same 60% for a tiny deal and a huge one, for a strong salesperson and a weak one, for a deal opened last week and one that has been sitting there eight months.
2. **Dead deals still count.** Acme has not replied to an email in six weeks. Everyone in the office knows it is dead. The spreadsheet still says 60%.
3. **People are biased.** Raj is an optimist and always promises more than he delivers. Priya is cautious and always beats her number. The formula treats them identically.
4. **Signed is not paid.** Acme signs on 30 March. We book Rs 500,000. But their payment terms are 60 days and they historically pay 20 days late, so the cash lands in late May. If the question was "how much money arrives in 30 days", that Rs 500,000 is a lie.

The one-line goal. Turn a messy pipeline into an honest revenue range for the next 30, 60 and 90 days — for both bookings and cash — and be able to explain every rupee of movement between two runs.

Who would use this and when

A sales leader opens it on Monday morning. They see a range, not a number. They see the probability of hitting the quarter target. They see which three deals carry most of the risk, and a list of what changed since last Monday with the reason next to each item. They know who to call before lunch.

Part 2 — How the system works

The three questions

For every single open deal, we answer three things:

1. **Will it actually close?** Not the invented stage percentage. A real probability built from deal size, customer segment, salesperson track record, how old the deal is, how long it has been silent, how long it has been stuck in its current stage, and how many times the close date has already been pushed.
2. **When will it close?** Not the date the salesperson typed in. A realistic date range based on how long similar deals really took, adjusted for seasonality and for how often deals at this stage slip.
3. **When does the money actually arrive?** Close date, plus payment terms, plus how late that particular customer usually pays.

The dice trick

Instead of calculating one number, we simulate ten thousand possible futures.

In one possible future: Acme rolls the dice and wins, closing 12 April, cash arriving 10 June. Globex rolls and loses, contributing nothing. Initech wins, closing 20 March, cash arriving 25 April. That future gives Rs 200,000 in the 30-day window.

Run that ten thousand times with different dice each time. Now sort the ten thousand results:

- the worst 10% of futures came in below a certain number — that is our **worst case**
- the middle value is our **expected case**
- the best 10% came in above another number — that is our **best case**

That is the whole engine. Everything else exists to make each dice roll more realistic.

Why dice instead of multiplication

Deals are all-or-nothing. A deal worth Rs 1,000,000 either lands or it does not. You never receive Rs 600,000 of it. Plain multiplication hides that lumpiness. Simulation shows it for free, which is exactly what makes the worst case meaningful.

The hard part, in plain words

Every week the forecast runs. Last week it said Rs 2,400,000. This week it says Rs 1,920,000. The only question a manager has is: **what happened?**

Most tools cannot answer this. Ours must. The output should read like a bank statement:

```
30-day forecast: 2,400,000 -> 1,920,000 (-480,000)

-310,000 Acme Corp      close date moved 28 Mar -> 15 Apr, fell outside window
-120,000 Globex         marked Closed Lost
-95,000  4 deals        no activity for 21+ days, probability decayed
-60,000  Raj Sharma     calibration factor 0.88 -> 0.79 after two misses
+105,000 Initech        new deal entered at Proposal stage
      0 interaction residual
```

Those lines must add up exactly to the total change. That is the feature nobody else will have, and it is what turns "we are going to miss" into "we are going to miss because Acme slipped — call Acme today".

Part 3 — Architecture

The nine layers

Data flows in one direction. Each layer is a separate Python module, so they can be built and tested independently.

#	Layer	What it does	Owner
1	Data generator	Invents 3 years of fake CRM history with hidden truth values we control	A
2	Database	SQLite. Deals plus a full event log of every stage change and date push	A
3	Feature builder	Turns raw events into numbers per deal: age, days silent, days in stage, times pushed	A
4a	Win probability model	Logistic regression giving a real probability per deal	A
4b	Timing model	Cycle length, slippage probability, seasonality	A
4c	Rep calibration	Compares each rep's past promises to actual results	A
4d	Payment delay model	How many days after signing the cash actually arrives	A
5	Simulation engine	10,000 trials rolling dice for every deal	B
6	Snapshot writer	Saves the complete state of every deal at every run	B
7	Attribution engine	Differs two snapshots, produces named causes that sum exactly	B
8	API	FastAPI endpoints serving all of the above	B

#	Layer	What it does	Owner
9	Dashboard	Four React screens	A and B split

Two things worth noticing about this shape. First, the four models in layer 4 are independent of each other, so they can be written in any order. Second, layers 6 and 7 are the distinctive part of the project, and they only work if layer 2 stored the event history properly. Layer 2 is not boring plumbing — it is the foundation everything stands on.

Tech stack — everything free, no accounts, no paid tiers

Layer	Tool	Licence	Why
Language	Python 3.11	PSF	All the statistical libraries live here
Database	SQLite	Public domain	A single file, no server, no install
Data work	pandas, NumPy	BSD	Tables and fast maths
Models	scikit-learn, SciPy	BSD	Logistic regression, distributions
Backend	FastAPI, Uvicorn	MIT	Automatic API docs, quick to write
Frontend	React, Vite, Tailwind	MIT	Standard, fast, looks good
Charts	Recharts	MIT	Waterfall, fan chart, histogram
Scheduling	APScheduler	MIT	Run the forecast nightly
Packaging	Docker Compose	Apache 2.0	One command starts everything
Version control	Git, GitHub	Free	Public repository

No cloud services. No API keys. No paid tiers. No LLM calls. The whole project runs on a laptop with one command.

Shortcut if time gets tight. Replace React with **Streamlit** (also free, also MIT). The dashboard then gets written in Python in roughly one day instead of three. You lose some polish. Decide this at the Day 7 checkpoint, not before.

Database schema

Eleven tables in four groups.

Group 1 — raw pipeline

```
customers(id, name, segment, payment_terms_days, region)
salespeople(id, name, team, hire_date)
deals(id, customer_id, salesperson_id, value, stage,
      expected_close_date, created_at, last_activity_date, status)
```

`segment` is one of SMB, Mid-Market, Enterprise. `status` is open, won or lost.

Group 2 — the event log (most important)

```
stage_events(id, deal_id, from_stage, to_stage, changed_at)
closedate_events(id, deal_id, old_date, new_date, changed_at)
activity_events(id, deal_id, type, occurred_at)
```

Every change is a new row. Nothing is ever overwritten. This is the only way to know that a deal has been pushed three times or has been silent for 24 days.

Group 3 — outcomes and cash

```
deal_outcomes(deal_id, result, actual_close_date, lost_reason)
payments(id, deal_id, invoice_date, due_date, paid_date, amount)
```

Group 4 — forecast records

```
forecast_runs(id, run_at, horizon_days, p10, p50, p90, mean,
              concentration_hhi, top3_share)

forecast_deal_snapshots(run_id, deal_id, value, stage, win_probability,
                        expected_close_date, expected_cash_date,
                        contribution_30, contribution_60, contribution_90,
                        factors_json)

forecast_attributions(run_id, prev_run_id, horizon_days,
                     cause_type, deal_id, amount, description)

overrides(id, deal_id, field, value, reason, created_by, created_at)
```

`forecast_deal_snapshots` is the key table for the hard part. It freezes the full state of every deal at the moment of each run, which is what gives the diff something concrete to compare.

Part 4 — The two pieces of real engineering

The simulation engine

The core is about fifteen lines. Everything else feeds it.

```

results_30, results_60, results_90 = [], [], []

for trial in range(10_000):
    r30 = r60 = r90 = 0
    for deal in open_deals:
        if random() < deal.p_win:
            # model 4a
            close = sample_close_date(deal)
            # model 4b
            cash = close + sample_payment_delay(deal)
            # model 4d
            days = (cash - today).days
            if days <= 30: r30 += deal.value
            if days <= 60: r60 += deal.value
            if days <= 90: r90 += deal.value
    results_30.append(r30)
    results_60.append(r60)
    results_90.append(r90)

worst = percentile(results_30, 10)
expected = percentile(results_30, 50)
best = percentile(results_30, 90)

```

Run it twice — once using close date to get **bookings**, once using cash date to get **collections**. Two curves on one chart.

Speed note. The loop above is clear but slow. Once it works correctly, rewrite it with NumPy: build a 10,000 x n_deals matrix of random numbers and compare it against the probability vector in one operation. Runtime drops from minutes to milliseconds. Do not optimise before it is correct.

The attribution engine

This is the part that makes the project worth building. The method is a **staged walk**: start from the old forecast, apply one category of change at a time, and record how far the number moves at each step.

```

def attribute(prev_snapshot, curr_snapshot, horizon):
    state = prev_snapshot.copy()
    baseline = forecast(state, horizon)
    causes = []

    for cause in CAUSE_ORDER:
        state = apply_changes_of_type(state, curr_snapshot, cause)
        new_value = forecast(state, horizon)
        causes.append({
            "cause": cause,
            "delta": new_value - baseline,
            "deals": deals_touched_by(cause),
        })
        baseline = new_value

    residual = forecast(curr_snapshot, horizon) - baseline
    causes.append({"cause": "interaction_residual", "delta": residual})
    return causes

```


The fixed order of causes

The order matters, because these factors interact with each other. Fix it once and never change it.

1. Deals closed won
2. Deals closed lost
3. New deals entered the pipeline
4. Deal values changed
5. Stage moves
6. Close-date changes (slippage)
7. Ageing and inactivity decay
8. Model retrain or calibration changes
9. Window shift (time simply passed)

Because the order is fixed, interaction effects leak into whichever step comes later. Whatever does not fit is captured as `interaction_residual` and displayed openly.

Keep the residual under 5% of the total change, and always show it. Being honest and slightly imprecise beats being fake-precise. Displaying the residual is itself a credibility signal — it tells the reader you know where the limits of the method are.

Part 5 — What we ship

Four screens

Screen	Contents	Built by
1. Forecast	Fan chart of the range over time. Big worst / expected / best numbers at 30, 60, 90 days. Toggle between bookings and cash. Gap to target with probability of hitting it.	B
2. What changed	Waterfall chart from last forecast to this one. Every bar clickable, expanding into the deals behind it. This is the demo screen.	B
3. Deal risk	Table sorted by value x probability of slipping or losing — not by deal size. Each row shows the reason in words.	A
4. Trust	Model accuracy over time, coverage rate, comparison against the naive baseline, plus the rep calibration scoreboard.	A

Metrics we report

Accuracy

- **MAPE by horizon.** The 30-day figure should beat the 90-day figure. If it does not, something is wrong.
- **Bias.** Signed mean error — are we systematically high or low?
- **Comparison against the naive baseline.** We must show that we beat plain `sum(value x stage_probability)`. This is the single most convincing number in the project.

Uncertainty quality — the sophisticated one

- **Coverage.** How often did the actual result land inside the P10 to P90 band? Target is 80%. Much higher means our bands are so wide they are useless. Much lower means we are overconfident.
- **Band width** relative to the size of the forecast.

Business

- Probability of hitting target
- Concentration: HHI and top-3 share
- Cash at risk of late collection

Explanation quality

- **Attribution residual** as a percentage of total change — target under 5%
- **Number of deals explaining 80% of the change** — usually 3 to 5

Part 6 — Dividing the work between two people

The principle

Split by **data flow position**, not by skill type. Member A owns everything that produces numbers about deals. Member B owns everything that consumes those numbers and turns them into a product. Each person builds the two dashboard screens that show their own work, so nobody waits on the other for the frontend.

	Member A — Data and Models	Member B — Engine and Product
Owens	Layers 1, 2, 3, 4a-4d, metrics	Layers 5, 6, 7, 8
Core skill	Statistics, pandas, scikit-learn	Systems, simulation, API, product thinking
Hardest task	Making the models genuinely predictive and validating against ground truth	The attribution engine
Screens	3 (Deal risk), 4 (Trust)	1 (Forecast), 2 (What changed)
Rough effort	10 working days	10 working days

The contract — read this before writing any code

The whole two-person plan depends on one file, written together on Day 1 and then frozen. It is the only thing the two halves of the project share.

```
# src/contract.py -- FROZEN after Day 1.
# Any change requires both members to agree, in writing, in the repo.

from typing import TypedDict
from datetime import date

class DealInput(TypedDict):
    deal_id: str
    value: float
    stage: str
    segment: str
    salesperson_id: str
    customer_id: str

    p_win: float          # 0..1, already adjusted for rep calibration
    cycle_mu: float        # lognormal mu: days from today to close
    cycle_sigma: float
    slip_prob: float       # 0..1, extra chance of pushing a period
    payment_delay_mu: float # lognormal mu: days from close to cash
    payment_delay_sigma: float

    factors: dict          # named contributions to p_win, for explanation
                          # e.g. {"late_stage": +0.18, "silent_24d": -0.22}

def get_deal_inputs(as_of: date) -> list[DealInput]: ...
def get_seasonality() -> dict[int, float]: ...      # month 1-12 -> multiplier
def get_rep_calibration() -> dict[str, float]: ...  # rep_id -> multiplier
```

The `factors` field is not optional decoration. It is how Member B's attribution engine is able to say "probability decayed because the deal went silent" instead of just "probability changed". Member A must populate it from the start, even with rough values.

How this unblocks both people on Day 1

Member B writes `src/fake_contract.py` on the first morning — a version of `get_deal_inputs` that returns fifty hard-coded deals with made-up but plausible values. Member B builds the entire simulation, snapshot and attribution engine against that fake. Member A builds the real implementation in parallel. On Day 5 they swap the import, and nothing else changes.

Neither person is ever blocked waiting for the other. This is the single most important design decision in the plan.

Part 7 — Member A: instructions

Your job in one sentence. Take raw deal records and turn them into honest numbers — a real win probability, a realistic close-date distribution, and a realistic cash-arrival distribution — then prove those numbers are good.

Day-by-day

Day 1 — Foundations

- Sit with Member B for one hour and write `src/contract.py` together. Freeze it.
- Write `src/db/schema.sql` with all eleven tables.
- Write `src/db/repo.py` — simple functions to read and write each table. No ORM, plain SQL is fine and faster to debug.
- **Deliverable:** an empty SQLite database that can be created from scratch with one command.

Day 2 — The data generator

This is more important than it sounds, so give it a full day.

- Write `src/generator/config.py` containing the **hidden truth**: the real win rate per segment, the real mean cycle length per segment and size band, the real optimism factor for each of six salespeople, the real seasonality multipliers per month, the real payment delay per customer.
- Write `src/generator/generate.py` which uses those parameters to invent three years of history plus roughly 300 currently open deals — including the full event log of stage changes and close-date pushes.
- **Deliverable:** a populated database, and a printed table of the hidden truth values saved to `docs/ground_truth.md`.

Why the hidden truth matters. Later, when you fit your models, you can check whether they recover the values you planted. If you set Enterprise cycle length to 94 days and your model estimates 91, your model works. Without this you have no way to know if anything you built is correct. This is your entire validation strategy.

Day 3 — Features

- Write `src/features/build.py`. For each deal, compute from the event log: deal age in days, days since last activity, days in current stage, number of close-date pushes, stage-velocity ratio against the segment median, log of deal value, segment one-hot, the rep's historical win rate.
- **Deliverable:** a function returning a clean pandas DataFrame, one row per deal.

Day 4-5 — Win probability model

- Fit a logistic regression on historical won/lost deals using those features.
- Produce a probability per open deal, and a `factors` dictionary giving each feature's contribution. For logistic regression this is just `coefficient x feature_value`, which is why we chose it over a black box.
- Check against ground truth and against the naive stage-probability baseline.
- **Deliverable:** `src/models/win_probability.py` plus a short note in the repo showing you beat the baseline.

Day 6 — Timing, slippage, seasonality

- Fit a lognormal cycle-length distribution per segment and size band.
- Compute slippage probability per stage from `closedate_events` — what fraction of deals at each stage had their date pushed, and by how many days on average.
- Compute seasonality multipliers per month from historical close counts.
- **Deliverable:** `src/models/timing.py`.

Day 7 — Calibration and payments

- `src/models/calibration.py`: for each salesperson, sum of what they committed versus sum of what they actually closed, over past periods. That ratio is their factor. Apply shrinkage toward 1.0 when a rep has few deals, so one unlucky quarter does not produce a factor of 0.2.
- `src/models/payment.py`: per-customer distribution of days between due date and paid date.
- **Deliverable:** both modules, plus `get_rep_calibration()` wired into the contract.

Day 8 — Wire up the contract and hand over

- Implement the real `get_deal_inputs()` against all four models.
- Meet Member B and swap their fake import for your real one. Fix whatever breaks — expect something to break here, and budget the day for it.
- **Deliverable:** Member B's engine runs on real data.

Day 9 — Metrics

- Write `src/metrics/accuracy.py`: MAPE by horizon, signed bias, coverage rate, and the naive-baseline comparison.
- Write the backtest: rewind to a past date, run the whole pipeline using only data known at that time, compare to what actually happened. Repeat for twelve past dates.
- **Deliverable:** a table of accuracy numbers you can put on a slide.

Day 10 — Screens 3 and 4

- Screen 3, Deal Risk: table sorted by `value x (1 - p_win)`, with a plain-English reason column generated from the `factors` dictionary.
- Screen 4, Trust: accuracy over time, coverage, baseline comparison, rep calibration scoreboard.

Definition of done for Member A

- Every model recovers its planted ground-truth parameter within a reasonable margin.
- The win probability model beats the naive stage baseline on historical data.
- `get_deal_inputs()` returns a valid `DealInput` for every open deal, with a populated `factors` dictionary.
- Coverage of the P10-P90 band is between 70% and 90% across the backtest.

Part 8 — Member B: instructions

Your job in one sentence. Take the per-deal numbers, turn them into a forecast range, remember every run, and be able to explain exactly why this run differs from the last one.

Day-by-day

Day 1 — Foundations and the fake

- Sit with Member A for one hour and write `src/contract.py` together. Freeze it.
- Set up the repository skeleton, `requirements.txt`, `docker-compose.yml`, and a `make run` style entry point.
- Write `src/fake_contract.py`: fifty hard-coded deals matching `DealInput` exactly, with plausible values spread across segments and sizes. Include two deliberately huge deals so concentration effects are visible.
- **Deliverable:** `python -m src.main` prints something, and the fake data loads.

Day 2 — The simulation, slow version

- Write `src/engine/simulate.py` exactly as in Part 4 — a plain Python loop, clear and obviously correct.
- Sample close dates from the lognormal parameters, apply the seasonality multiplier, apply the slippage draw.
- Compute cash date and bucket into the 30, 60 and 90 day windows.
- **Deliverable:** printed P10, P50, P90 for all three horizons, for both bookings and cash. The project now works end to end on day two.

Day 3 — Make it fast and add risk measures

- Rewrite the loop with NumPy matrices. Verify the fast version gives the same answers as the slow one within noise — keep the slow one as a test.
- Add concentration measures: HHI, top-3 share, and "if the largest deal slips, the forecast drops by X%".
- Add probability of hitting a target, which is simply the fraction of trials above that target.
- **Deliverable:** a full run in under two seconds.

Day 4 — Snapshots

- Write `src/engine/snapshot.py`. After every run, write one row to `forecast_runs` and one row per deal to `forecast_deal_snapshots`, including the `factors_json`.
- Snapshots are immutable. Never update one, only insert.
- Add a small CLI to run the forecast for a given as-of date, so you can generate a sequence of historical runs to test against.
- **Deliverable:** run the forecast three times with small changes in between, and see three complete snapshots in the database.

Day 5-6 — The attribution engine

This is the centrepiece of the project. Give it two full days and do not let anything eat into them.

- Write `src/engine/attribution.py` implementing the staged walk from Part 4.
- Implement each of the nine cause categories as a function that transforms the old state one step toward the new state.
- Compute the residual and include it in the output.
- Write `tests/test_attribution.py` with these tests:
 - the causes always sum to the total change, within rounding
 - the residual stays under 5% of the total change
 - when you artificially push exactly one deal's close date and rerun, the engine names that deal and no other
 - when you mark one deal lost, the engine attributes almost exactly that deal's contribution
- **Deliverable:** a passing test suite and a printed attribution table.

Day 7 — API

- Write `src/api/main.py` with FastAPI. Endpoints:
 - `GET /forecast?horizon=30&basis=cash` — the range and the distribution
 - `GET /forecast/changes?run_id=...` — the attribution list
 - `GET /deals/risk` — for Member A's screen 3
 - `GET /metrics/accuracy` — for Member A's screen 4
 - `POST /run` — trigger a forecast run
- Agree the exact JSON shapes with Member A before writing the frontend.
- **Deliverable:** FastAPI's automatic docs page shows every endpoint working.

Day 8 — Swap in real data

- Replace `fake_contract` with Member A's real implementation. Expect breakage; this is normal and the day is budgeted for it.
- Sanity-check the outputs: are the probabilities distributed sensibly, are the close dates plausible, does the range look reasonable against the pipeline total?
- **Deliverable:** the forecast runs on the real generated dataset.

Day 9-10 — Screens 1 and 2

- Screen 1, Forecast: fan chart, the big three numbers per horizon, bookings and cash toggle, gap to target with probability of hitting it.
- Screen 2, What changed: waterfall chart from previous run to current. Each bar clickable, expanding to show the deals behind it. Spend the extra time here — this is the screen that wins the demo.

Definition of done for Member B

- A full forecast run completes in under two seconds.
- Attribution causes sum to the total change and the residual stays under 5%.
- The single-deal injection test passes: push one deal, and the engine names it.
- The waterfall screen renders from real data and every bar opens to reveal its deals.

Part 9 — Working together

Daily sync, fifteen minutes

Same time every day. Three questions only: what did you finish, what are you doing today, what are you blocked on. If something is blocked, fix it in the next fifteen minutes or write it down as a risk.

The five checkpoints that actually matter

Day	Checkpoint	Both must agree
1	Contract frozen	<code>src/contract.py</code> is committed and neither person will change it alone
3	Both halves running independently	A has real data in the database, B has a working forecast on fake data
6	JSON shapes agreed	Every API response format is written down before any frontend work starts
8	Integration	The fake is swapped for the real implementation and the whole pipeline runs
10	Demo rehearsal	Walk the four screens end to end, out loud, twice

Repository layout

```
revenue-forecast/
  docker-compose.yml
  requirements.txt
  README.md
  data/
    crm.db                # SQLite file, gitignored
  docs/
    ground_truth.md      # A: the planted parameters
    api_shapes.md         # both: agreed JSON formats
  src/
    contract.py           # BOTH - frozen Day 1
    fake_contract.py      # B - temporary, deleted Day 8
    generator/            # A
      config.py
      generate.py
    db/                   # A
      schema.sql
      repo.py
    features/build.py      # A
    models/               # A
      win_probability.py
      timing.py
      calibration.py
      payment.py
    engine/               # B
      simulate.py
      snapshot.py
      attribution.py
    metrics/accuracy.py   # A
    api/main.py           # B
  frontend/              # split: A does screens 3-4, B does 1-2
  tests/
    test_attribution.py   # B
    test_models.py        # A
```

Git rules to avoid pain

- One branch each: `member-a` and `member-b`. Merge to `main` at each checkpoint.
- Nobody edits files in the other person's directory. If you need a change there, ask.
- `src/contract.py` changes require both people present.
- Commit at least twice a day so the other person can see progress.

Part 10 — Claude session prompts

Paste the whole block below into a fresh Claude session at the start of your work. It gives Claude the full project context so you do not have to re-explain anything. Re-paste it if you start a new session later.

For Member A

I am building a CRM revenue forecasting engine with one teammate. I own the data and modelling half. Here is the full context.

PROJECT We are replacing the standard CRM forecast, which is just `sum(deal_value x fixed_stage_probability)`, because it inherits every bias in the pipeline: invented stage percentages, dead deals still counted, optimistic and sandbagging salespeople treated identically, and booked revenue confused with collected cash.

Our system answers three questions per deal - will it close, when will it close, when does the cash arrive - then runs a 10,000-trial Monte Carlo simulation to produce best, expected and worst case revenue for 30, 60 and 90 day horizons. It saves a snapshot every run, so it can explain exactly which deals and factors caused the forecast to change between runs. That explanation is the headline feature.

CONSTRAINTS Everything must be free and open source. No paid tiers, no cloud accounts, no API keys, no LLM calls. Stack is Python 3.11, SQLite, pandas, NumPy, scikit-learn, SciPy, FastAPI, React with Vite and Tailwind, Recharts, Docker Compose. The whole thing runs locally.

MY SCOPE - I own these

1. Data generator: three years of fake CRM history plus ~300 open deals, generated from a config file of hidden true parameters (real win rates per segment, real cycle lengths, real per-rep optimism factors, real seasonality, real payment delays). The hidden truth is our validation strategy - my models must recover the values I planted.
2. Database: SQLite, eleven tables. Critically an append-only event log - `stage_events`, `closedate_events`, `activity_events` - because deal age, silence, time-in-stage and number of date pushes all come from it.
3. Feature builder: turns the event log into a pandas DataFrame, one row per deal, with deal age, days since last activity, days in current stage, number of close-date pushes, stage velocity versus segment median, log deal value, segment, rep historical win rate.
4. Win probability model: logistic regression, chosen over a black box specifically because coefficient `x feature_value` gives me per-feature contributions I must expose.
5. Timing model: lognormal cycle length per segment and size band, slippage probability per stage derived from `closedate_events`, seasonality multipliers per month.
6. Rep calibration: committed versus actual per salesperson, with shrinkage toward 1.0 for reps with few deals.
7. Payment delay model: per-customer distribution of days between due date and paid date.
8. Accuracy metrics: MAPE by horizon, signed bias, P10-P90 coverage rate, and comparison against the naive stage-probability baseline. Plus a backtest that rewinds to past dates using only data known then.
9. Dashboard screens 3 (deal risk table sorted by value `x` probability of loss, with plain-English reasons) and 4 (accuracy over time plus rep calibration scoreboard).

MY TEAMMATE OWNS The Monte Carlo simulation engine, the snapshot writer, the attribution engine that diffs two snapshots, the FastAPI layer, and dashboard screens 1 (forecast fan chart) and 2 (waterfall of what changed).

THE CONTRACT BETWEEN US - frozen, I must not change it alone I implement `get_deal_inputs(as_of: date) -> list[DealInput]`, where `DealInput` is a TypedDict with: `deal_id`, `value`, `stage`, `segment`, `salesperson_id`, `customer_id`, `p_win` (already adjusted for rep calibration), `cycle_mu`, `cycle_sigma`, `slip_prob`, `payment_delay_mu`, `payment_delay_sigma`, and `factors` (a dict of named contributions to `p_win`, for example `{"late_stage": 0.18, "silent_24d": -0.22}`). I also implement `get_seasonality()` returning month 1-12 to multiplier, and `get_rep_calibration()` returning `rep_id` to multiplier. The `factors` dict is mandatory - my teammate's attribution engine uses it to say "probability decayed because the deal went silent" rather than just "probability changed". My teammate builds against a fake implementation of this contract until day 8, so neither of us is ever blocked.

HOW I WANT YOU TO HELP Work with me module by module in the order above. Before writing code for a module, briefly confirm the approach with me. Prefer plain SQL over an ORM. Keep functions testable and pure where possible. When you fit a model, always show me the check against the planted ground truth and against the naive baseline. Flag it clearly if you think a modelling choice will make my `factors` dict harder to produce, because that would break my teammate's work.

Start by helping me write the database schema and the generator config with hidden truth parameters.

For Member B

I am building a CRM revenue forecasting engine with one teammate. I own the engine and product half. Here is the full context.

PROJECT We are replacing the standard CRM forecast, which is just $\text{sum}(\text{deal_value} \times \text{fixed_stage_probability})$, because it inherits every bias in the pipeline: invented stage percentages, dead deals still counted, optimistic and sandbagging salespeople treated identically, and booked revenue confused with collected cash.

Our system answers three questions per deal - will it close, when will it close, when does the cash arrive - then runs a 10,000-trial Monte Carlo simulation to produce best, expected and worst case revenue for 30, 60 and 90 day horizons. It saves a snapshot every run, so it can explain exactly which deals and factors caused the forecast to change between runs. That explanation is the headline feature and I own it.

CONSTRAINTS Everything must be free and open source. No paid tiers, no cloud accounts, no API keys, no LLM calls. Stack is Python 3.11, SQLite, pandas, NumPy, scikit-learn, SciPy, FastAPI, React with Vite and Tailwind, Recharts, Docker Compose. The whole thing runs locally.

MY SCOPE - I own these

1. Repository skeleton, requirements, Docker Compose, entry point.
2. A fake implementation of our shared contract - fifty hard-coded deals - so I can build everything before my teammate's real data exists. Two of them deliberately huge so concentration effects show up.
3. Simulation engine: 10,000 trials. Per trial, per deal, roll against p_{win} ; if it wins, sample a close date from lognormal cycle parameters, apply the seasonality multiplier and a slippage draw, then add the payment delay to get a cash date, then bucket into 30, 60 and 90 day windows. Output P10, P50, P90. Run it twice - once on close date for bookings, once on cash date for collections. Write the slow clear loop first, then vectorise with NumPy and keep the slow one as a test.
4. Risk measures: HHI concentration, top-3 share, impact if the largest deal slips, and probability of hitting a target (the fraction of trials above it).
5. Snapshot writer: after every run, one row in `forecast_runs` and one row per deal in `forecast_deal_snapshots` including a `factors_json` column. Snapshots are immutable - insert only, never update. Plus a CLI to run a forecast for a given as-of date so I can generate a sequence of historical runs.
6. **ATTRIBUTION ENGINE** - the centrepiece. It diffs the previous snapshot against the current one using a staged walk: start from the old forecast, apply one category of change at a time, recompute, and record the delta at each step. Fixed cause order, never reordered: closed won, closed lost, new deals, value changes, stage moves, close-date changes, ageing and inactivity decay, model or calibration changes, window shift. Whatever does not fit is reported openly as `interaction_residual`, which must stay under 5% of the total change. The causes must sum exactly to the total change.
7. FastAPI endpoints: GET `/forecast` with horizon and basis params, GET `/forecast/changes`, GET `/deals/risk`, GET `/metrics/accuracy`, POST `/run`.
8. Dashboard screens 1 (fan chart, big three numbers per horizon, bookings and cash toggle, gap to target with probability of hitting it) and 2 (waterfall from previous run to current, every bar clickable to reveal the deals behind it).

MY TEAMMATE OWNS The fake CRM data generator with planted ground-truth parameters, the SQLite schema and event log, feature engineering, the win probability model (logistic regression), the timing and slippage model, rep calibration, the payment delay model, accuracy metrics and backtesting, and dashboard screens 3 and 4.

THE CONTRACT BETWEEN US - frozen, I must not change it alone My teammate implements

`get_deal_inputs(as_of: date) -> list[DealInput]`, where `DealInput` is a TypedDict with: `deal_id`, `value`, `stage`, `segment`, `salesperson_id`, `customer_id`, `p_win` (already adjusted for rep calibration), `cycle_mu`, `cycle_sigma`, `slip_prob`, `payment_delay_mu`, `payment_delay_sigma`, and `factors` (a dict of named contributions to p_{win}). Also `get_seasonality()` and `get_rep_calibration()`. The `factors` dict is what lets my attribution engine name model-factor causes in plain English rather than just reporting that a probability moved. I consume it, I do not produce it. I build against my fake implementation until day 8, then swap the import.

TESTS I MUST HAVE FOR THE ATTRIBUTION ENGINE

- causes always sum to the total change, within rounding
- residual stays under 5% of the total change
- if I artificially push exactly one deal's close date and rerun, the engine names that deal and no other
- if I mark one deal lost, the engine attributes almost exactly that deal's contribution

HOW I WANT YOU TO HELP Work with me module by module in the order above. Before writing code for a module, briefly confirm the approach with me. Correctness before speed - I want the obvious slow version working before any vectorisation. Keep the simulation deterministic under a fixed random seed so my tests are stable. Push back if you see a design choice that would make the attribution engine harder to keep additive, because that is the feature the whole project is judged on. Start by helping me set up the repository skeleton and write the fake contract implementation.

Part 11 — Risks and what to cut

The main risk

Spending all the time on the models and running out of time for attribution. The models are the familiar, comfortable part and they will pull attention. Attribution is the part nobody else will have. **Protect Member B's Day 5 and 6 absolutely.** Nothing else goes into those two days.

Other risks and their fixes

Risk	Fix
The contract changes midway and breaks both halves	Freeze it Day 1. Changes need both people present.
Integration on Day 8 takes longer than a day	Have Member B run against a small real extract on Day 5 as a smoke test
The frontend eats the last three days	Decide Streamlit versus React at the Day 7 checkpoint, based on actual progress
The generated data is too clean, so models look perfect	Deliberately add noise, missing values and a few irrational deals to the generator
Attribution residual will not come down	Accept it and display it. A visible 4% residual is better than a hidden fudge.

If you must cut, keep exactly these four

1. **The waterfall attribution** — what changed and why
2. **The distribution curve with probability of hitting target** — honest uncertainty
3. **The risk leaderboard with reasons** — what to do about it
4. **The coverage metric** — proof the bands are trustworthy

Those four tell the complete story: what, why, act, trust. Everything else is supporting detail.